

Lec8 Inheritance

- construct的时候, 先调用base, 后调用derived; deconstruct的时候相反

```
using Animal::Animal;
```

表示直接继承基类的constructor

C++

dynamic binding / virtual dispatching

```
class Animal {
public:
    Animal(const string& name, int age): name(name), age(age) {}
    virtual void make_sound() {
        cout << "Animal " << name << " makes sound.\n";
    }
protected:
    string name;
    int age;
};

class Dog : public Animal {
public:
    using Animal::Animal;
    void make_sound() override {
        cout << "Dog " << name << " barks.\n";
    }
};
```

C++

- 如果一个函数在base里面被声明为virtual, 它可以被derived重写。
 - 当通过base的pointer/ reference 调用的时候, 程序会在运行的时候决定调用谁

```
Animal* p = new Dog("dog1",3);
p->make_sound();
```

C++

- p 的静态类型是Animal*, 但是实际指向的是Dog对象
- 因为 `make_sound()` 是虚函数, 所以最后调用的是 `Dog::make_sound()`
 - called **polymorphism**, 同一个函数在不同派生类中有不同形式

- 派生类中包含一个完整的base类
 - it's always safe to thread a derived class as a base class
 - `Cat& cat_ref1 = dynamic_cast<Cat&>(animal_ref1)` 有问题。没法把一个base类ref转化为Cat类
 - `Cat* cat_ptr1` 同理

Hiding inherited functionality

Changing an inherited member's access level

```
class Base{
private :
    int m_value{};
public:
    Bass (int value)
        : m_value(value)
    {}

protected:
    void printValue() const {std::cout << m_value;}
};

class Derived : public Base
{
public :
    Derived(int value) : Base {value}
    {}

    using Base::printValue; ← 可以把base的printValue在derived类中设置为public
}
```

C++

Hiding functionality

- 在derived中可以隐藏base中存在的功能，使其无法通过derived访问->设置为private

Deleting functions in the derived class

```
class Derived : public Base{
public :
    Derived(int value)
        : Base {value}
```

C++

```
{}
```

```
int getValue() const = delete;
```

- derived的getValue被删除，但是base的getValue仍然存在
- 访问方式：

```
derived.Base::getValue();  
static_cast<Base&>(derived).getValue(); ← 把derived当成他里面的base部分来看。
```

C++

object slicing

```
Cat cat("cat",3);  
Animal animal = cat;
```

C++

- 合法，但是cat里属于animal的部分被copy进了animal，其余的被删掉，animal完全是一个Animal对象
- 所以，不要copy或者moving polymorphic objects。用指针或引用

destructor

- 当一个类要被 **polymorphic** 地使用时，**base class 的 destructor 要写成 virtual**
 - 这里的“polymorphic 使用”指的是：`cpp A* p = new B; delete p;`
- 如果 **A** 的 destructor **不是 virtual**，那么通过 **A*** 删除实际的 **B** 对象时，**B** 的 destructor 可能不会被正确调用，导致 **B** 中管理的资源无法释放
- 注意 **virtual destructor 不是为了让 B 析构时调用 A**。
 - 只要一个 **B** 对象正常析构，析构顺序本来就一定是 `~B() ~A()` virtual 解决的是：
通过 base pointer 删除 derived object 时，能否正确从 derived destructor 开始析构
- 正确写法：

```
class A {  
    public: A() {}  
    virtual ~A() {}  
};  
class B : public A {
```

C++

```
public: B() {} ~B() {}  
};
```

- 当执行：

```
A* p = new B; delete p;
```

- 因为 **A** 的 destructor 是 **virtual**，所以会正确调用：

```
~B() ~A()
```

C++

- 另外，**A** 的 destructor 一旦是 **virtual**，**B** 的 destructor 会自动也是 virtual，因此：

```
~B() {}
```

C++

就够了，不一定要再写：

```
virtual ~B() {}
```

场景	A 的 destructor 是否需要 virtual	析构结果	
B b;	不需要	~B() → ~A()	
B* p = new B; delete p;	不需要	~B() → ~A()	
A* p = new B; delete p;	需要	有 virtual ：~B() → ~A()；没有 virtual ：可能出问题	...

abstract class

- 如果还不知道base类里面要实现什么，可以用抽象类（只有纯虚函数）
 - 不能直接创造object
 - derived class **必须** 实现纯虚函数，否则自己还是abstract class